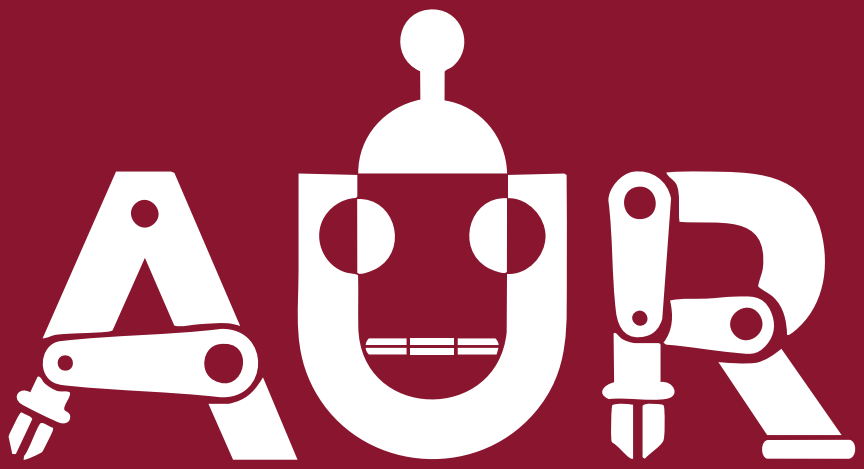

Training '26

Software | Phase II

ROS2 workshop 1

Published: Friday 4th, September 2026

Revision

Before we dive into the tasks, let's review the fundamental concepts you will be using today. You already know that ROS 2 nodes communicate using **Topics**, **Services**, and **Actions**, and you are familiar with **Parameter Files** and **Launch Files**. Here is a quick refresher on the components we will use:

- **Service Communication**:
  - **Service Client**: Sends a **Request** to a service server and waits for a **Response**.
  - **Service Server**: Receives the request, performs quick synchronous processing, and returns a response.
- **Action Communication**:
  - **Action Client**: Sends a **Goal** to an action server; optionally subscribes to periodic **Feedback**, receives a final **Result**, and can cancel the goal at any time.
  - **Action Server**: Receives the goal, executes long-running or non-blocking tasks while publishing feedback, and returns a final result upon completion.
- **Parameter Files**: Used to configure node inputs externally (typically via YAML). They eliminate hardcoded values, simplifying testing across different operational scenarios.
- **Launch Files**: Execute and manage multiple nodes, parameter files, and system dependencies simultaneously through a single execution command.

Task 1: Parameterize Last Workshop's Task

Goal: Convert your proportional control node from Workshop 1 (Task 2) to use a YAML parameter file instead of hardcoded values. The parameters to externalize are:

- **Target Coordinates:** `target_x`, `target_y`
- **Control Gains:** `linear_gain`, `angular_gain`
- **Tolerance:** `distance_tolerance`, `angle_tolerance`
- **Loop Rate:** `loop_rate_hz`

Task 2: Services Toggle

Goal:

- Change the behavior of the last workshops code by making it a Service Server using `std_srvs/srv/SetBool`.

-> When a client sends `data: True`, the node starts publishing data to make turtlsime move to the target coordinates.
-> reponse with `success: True` if the request was accepted.

- Create a Service Client that calls the service to start the turtle's movement after a time delay passes

> **Tip**
>
> A `SetBool` callback is expected to return quickly. Don't read serial data inside it — the executor will block and your node will appear frozen for the duration. Instead, have the callback only flip a boolean flag (e.g. `self.reading_active`), and do the actual serial reads in a separate mechanism: a fast `create_timer()` callback that checks the flag and reads one chunk each tick. Either is fine as long as the `SetBool` callback itself just toggles state and returns.

> ⋆ **Bonus**
>
> Instead of having the service use the `SetBool` change the service type using a **custom service** type that has a request with the target coordinates - *x: float, y: float*. The server will then use the request to set the target coordinates and start moving the turtle to that location. The service response can be a boolean indicating if the request was accepted or not (e.g. if the coordinates are out of bounds).
> **Notice 1**: The servoce itself must not block for the duration of the turtle's movement to the target coordinates. It should return immediately after setting the target coordinates and starting the movement; it returns the if goal is accepted or not; not if the turle reached the distination or not. This should have been implemented as an action, but it will be harder; details for action is given at the cancelled task at the end of this workshop.
> **Notice 2**: the x and y parameters in the parameter file will be ignored if you implement this bonus task.

Task 3: Launch File Integration

Goal: Write a Python launch file that executes `turtlesim_node`, service client that calls the service and your go to goal node (including loading its YAML parameter file) using a single command.

Task 4: Serial Communication

Goal: Write a ROS 2 node that reads sensor data from a microcontroller (e.g. Arduino) over a serial port and logs the data.

No Arduino on hand? Simulate one: write a tiny node that writes fake sensor lines to a virtual serial pair (`socat -d -d pty,raw,echo=0 pty,raw,echo=0` creates two linked `/dev/pts/N` devices), and point your node at one end. This keeps the task's logic identical without requiring hardware.

Node Code:

Python

```
"""
Writes fake sensor readings to a serial port once per second.

Use this with `socat` to simulate a microcontroller when no Arduino is
available:

    socat -d -d pty,raw,echo=0 pty,raw,echo=0

That prints two linked devices, e.g. /dev/pts/3 and /dev/pts/4. Run this
script pointed at one of them, and set serial_service_server's 'serial_port'
parameter to the other.

Usage:
    ros2 run workshop2 mock_sensor_source /dev/pts/3 9600
"""
import random
import sys
import time
import serial

def main():
    port = sys.argv[1] if len(sys.argv) > 1 else '/dev/pts/1'
    baud = int(sys.argv[2]) if len(sys.argv) > 2 else 9600

    conn = serial.Serial(port, baud, timeout=1.0)
    print(f'Writing fake sensor data to {port} at {baud} baud. Ctrl+C to stop.')
    try:
        while True:
            reading = round(random.uniform(20.0, 30.0), 2)
            line = f'TEMP:{reading}\n'
            conn.write(line.encode('utf-8'))
            time.sleep(1.0)
    except KeyboardInterrupt:
        pass
    finally:
        conn.close()
```

Cancelled Task: Waypoint Navigation (Action Interface)

Goal: Write a ROS 2 Action Server and Client to navigate the turtle to a specific target coordinate (x, y) on the screen.

> **Tip**
>
> You can reuse and adapt the proportional control logic written in Workshop 1 (Task 2) by encapsulating it inside an Action Server execution callback: publish `Feedback` where you used to just track state, and return a `Result` where the loop used to silently stop.

> **Tip**
>
> A custom `.action` file needs its own interface package, separate from the package containing your node code (e.g. `turtle_interfaces`). Build and source that package **before** the node package that imports it, or the import will fail.

1 How It Works

- **Action Structure:** Define or use an action interface containing:
  - **Goal:** `float32 x, float32 y`
  - **Feedback:** `turtlesim/Pose current_pose`
  - **Result:** `bool success`
- **Action Client:** Sends goal coordinates (x, y) to the server and waits for execution feedback and final result.
- **Action Server:** Accepts or rejects incoming goals, executes the proportional loop while periodically publishing the current turtle pose as feedback, and returns `success = True` once within distance tolerance.

2 Goal Validation

Don't accept every goal blindly — reject and log a warning for any (x, y) outside turtlesim's drawable window (approximately `0.0` to `11.0` on both axes). Use the goal request callback (`goal_callback`) to do this rejection **before** execution begins, not inside the execute callback.

Caution

You will have a concurrency problem; we did not cover concurrency yet; hence, this task is cancelled but left for reference.

Explanation of the Concurrency Problem:

You will need two callbacks: one for the execution of the goal and one that subscribes to the turtle's pose topic.
If no handeled concurrency is implemented, the execution callback will block the subscription callback from being called; hence, you will never be able to receive the pose updates and the turtle will never move.

How to handle the concurency:

You have two options:

1. Use a multi-threaded executor to allow the subscription callback to be called while the execution callback is running.
2. use `async` and `await` to handle the asynchronous nature of the execution callback.

you are not required to implement this task, but you can try it if you want to challenge yourself. The solution will be provided in the repo. Knowledge learnt from learning this task will be rewarding if you want to give it a try.